This CVPR 2020 paper is the Open Access version, provided by the Computer Vision Foundation.
Except for this watermark, it is identical to the accepted version;
the final published version of the proceedings is available on IEEE Xplore.

PF-Net: Point Fractal Network for 3D Point Cloud Completion

Zitian Huang[1], Yikuan Yu[1,2], Jiawen Xu[1], Feng Ni[2], Xinyi Le[1*]

[1]Shanghai Jiao Tong University [2]SenseTime

lexinyi@sjtu.edu.cn *

Abstract

In this paper, we propose a Point Fractal Network (PF-Net), a novel learning-based approach for precise and high-fidelity point cloud completion. Unlike existing point cloud completion networks, which generate the overall shape of the point cloud from the incomplete point cloud and always change existing points and encounter noise and geometrical loss, PF-Net preserves the spatial arrangements of the incomplete point cloud and can figure out the detailed geometrical structure of the missing region(s) in the prediction. To succeed at this task, PF-Net estimates the missing point cloud hierarchically by utilizing a feature-points-based multi-scale generating network. Further, we add up multi-stage completion loss and adversarial loss to generate more realistic missing region(s). The adversarial loss can better tackle multiple modes in the prediction. Our experiments demonstrate the effectiveness of our method for several challenging point cloud completion tasks.

1. Introduction

3D vision has become one of the current research topics recently. Among various types of 3D data description, point cloud has been widely used in 3D data processing due to its small data size but more delicate presenting ability [1–3]. Real-world point cloud data is often captured by using laser scanners, stereo cameras, or low-cost RGB-D scanners. However, due to occlusion, light reflection, transparency of surface material, and limitations of sensor resolution and viewing angle, it will cause a loss of geometric and semantic information, resulting in incomplete point clouds. Therefore, it is an essential task to repair incomplete point clouds for further applications. Since the 3D point cloud is unstructured and unordered, the majority of the deep-learning-based methods for dealing with 3D data transform point cloud to collections of images(e.g., views) or regularly voxel-based representations of 3D data. However, multi-view and voxel-based representation [4–9] leads to unnecessarily voluminous and limits the output resolution of voxels [10, 11].

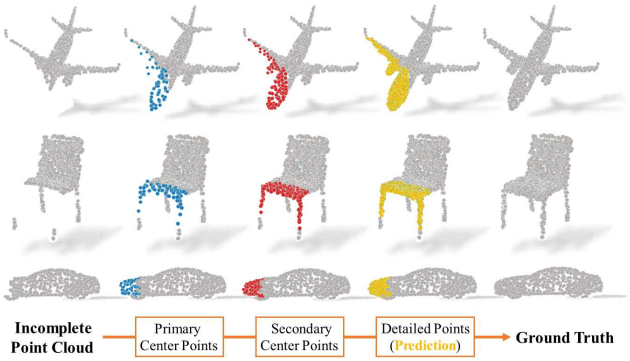


Figure 1. Our PF-Net takes the incomplete point cloud as input and directly predicts the missing point cloud. The display of multi-scale predictions (indicated by blue, red and yellow) are made by layers from different depths of Point Pyramid Decoder (PPD). The low resolution prediction serves as the center point to generate high resolution prediction, leading to a coarse-to-fine prediction.

Thanks to the advent of PointNet [10], learning-based architectures are capable of directly operating on point cloud data. L-GAN [12] introduces the first deep-learning-based network for point cloud completion by utilizing an Encoder-Decoder framework. PCN [11] combines the advantages of L-GAN [12] and FoldingNet [13] which is specialized on repairing incomplete point clouds. Recently, RL-GAN-Net [14] proposes a reinforcement learning agent controlled GAN for reducing prediction time of point cloud completion.

Previous works for completion take incomplete point cloud as input and aim to output the overall complete models. They pay more attention to learning the general character of a genus/category but not the local details of a specific object. Therefore, they may change the position of known points, and encounter genus-wise distortions [13], which causes noise and detailed geometrical loss. For example, in Fig. 6(3), the reconstruction of a chair in previ-

*The work described in the paper was jointly sponsored by Natural Science Foundation of Shanghai (18ZR1420100) and National Natural Science Foundation of China (61703274).

ous works miss the existing cross-bar (under the chair), and fail to generate the hollow back. The reason may be that Auto-Encoders tend to average multiple "chairs" in the prediction. Noticing that most chairs in the training set have fully filled back without cross-bar, as is shown in Fig. 1, the previous networks thus are more likely to generate normal chairs with full filled back other than "special" chairs with hollow back and cross-bars in Fig. 6(3). In this work, we present the Point Fractal Network (PF-Net) for repairing the incomplete point cloud. Different from the existing Auto-Encoder architectures, the main features of PF-Net can be summarized as follows:

(1) To preserve the spatial arrangements of the original part, we take the partial point cloud as input and only output the missing part of the point cloud instead of the whole object. This architecture has two benefits: Firstly, it retains the geometric features of the original point cloud after repairing. Secondly, it helps the network focus on perceiving the location and structure of missing parts. The main difficulty of such kind of prediction is that partial known points and points to be inferred are merely related in semantic level and completely different in spatial arrangements.

(2) For better feature extraction of the specific partial point cloud, we first propose a novel Multi-Resolution Encoder (MRE) to extract multi-layer features from the partial point cloud and its low resolution feature points by utilizing a new feature extractor Combined Multi-Layer Perception (CMLP). These multi-scale features contain both local and global features, also low-level and high-level features, enhancing the ability of the network to extract semantic and geometric information.

(3) To tackle the genus-wise distortions problem [13] and generate fine-grained missing region(s), we design a Point Pyramid Decoder (PPD) to generate the missing point cloud hierarchically. PPD is a multi-scale generating network based on feature points. PPD will predict primary, secondary, and detailed points from layers of different depth. Primary points and secondary points will try to match their corresponding feature points and serve as the skeleton center points to propagate the overall geometry information to the final detailed points. Fig. 1 visualizes the process of this hierarchical prediction, which shows excellent performance on generating a high-quality missing region point cloud and restoring the original detailed shape.

In addition, we propose a multi-stage completion loss to guide the prediction of our method. Multi-stage completion loss helps the network pay more attention to the feature points. To further alleviate the distortions problem caused by Auto-Encoder framework, inspired by a renowned 2D context encoder [15], we reduce this burden in the loss function by jointly train our PF-Net to minimize both multi-stage completion loss and adversarial loss. Adversarial loss optimizes our network that enable it to select a specific point cloud from multiple modes.

2. Related Work

2.1. Shape Completion and Deep Learning

Currently, 3D shape completion methods are mainly based on the 3D voxel grid and point cloud. For voxel grid-based algorithm, architectures such as 3D-RecGAN [16], 3D-Encoder-Predictor Networks (3D-EPN) [17], and hybrid framework of 3D-ED-GAN and LRCN [18] have been developed to realize the goal of repairing incomplete input data. However, voxel-based methods are restricted by its resolution because the computation cost increases dramatically with resolution.

PointNet [10] solves the problem of processing unordered point sets. Thus, algorithms which can directly conduct shape completion task on unordered point have been significantly developed [19–23]. L-GAN [12] introduces the first deep generative model for the point cloud. Though L-GAN [12] is capable of conducting shape completion tasks to some extent, its architecture is not primarily built to do shape completion task, and thus the performance is not considered ideal. FoldingNet [13] introduces a new decoding operation named Folding, serving as a 2D-to-3D mapping. Later, Point Completion Network (PCN) [11] proposes the first learning-based architecture focusing on shape completion task. PCN [11] applies the Folding operation [13] to approximate a relatively smooth surface and conduct shape completion. Recently, a reinforcement learning agent controlled GAN based network (RL-GAN-Net) [14] is invented for real-time point cloud completion. An RL agent is used in [14] to avoid complex optimization and accelerate prediction process, but it does not focus on enhancing prediction accuracy of the points. 3D point capsule networks [24, 25] surpasses the performance of other methods and becomes the state-of-the-art Auto-Encoder processing point cloud, especially in the domain of point cloud reconstruction.

2.2. Context Encoder and Feature Pyramid Network

Context encoder [15] processes images with missing region and trains a convolutional neural network to regress the missing pixel values. This architecture is formed by an encoder capturing the unspoiled content of the image and a decoder to predict the missing image content. In addition, it jointly train reconstruction and adversarial loss for semantic inpainting.

Feature Pyramid Network (FPN) [26] is an efficient method to solve the problem of multi-scale feature extraction. Thanks to the architecture of FPN [26], the final feature map is the fusion of both semantically-rich and locally-rich features [27–31]. It enhances the geometric and seman-

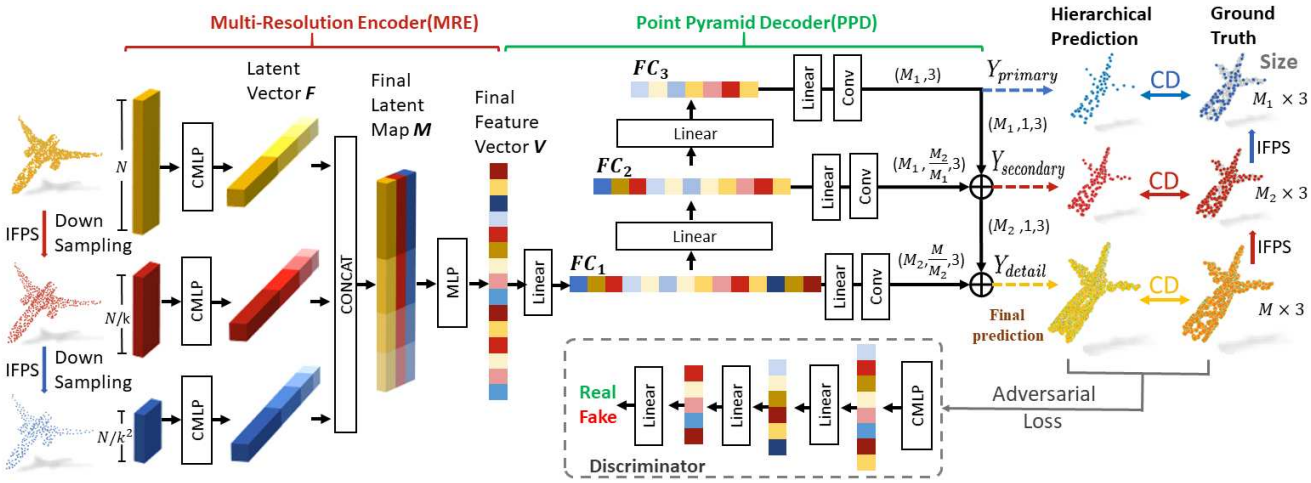


Figure 2. **The architecture of PF-Net.** The input of PF-Net is incomplete point cloud and it predicts the missing point cloud by utilizing a multi-scale generating network PPD. IFPS (Section 3.1) samples the input (indicated by yellow) into different scales (indicated by red and blue). Three scales of prediction are indicated by blue, red and yellow (final prediction) in the figure. CMLP (Section 3.2) is our feature extractor. CD (Chamfer Distance in Section 3.4) is applied to evaluate the difference between the prediction and the ground truth.

tic information contained in the final feature map. FPN [26] partially inspires the decoder in our network.

3. Method

In this section, we will introduce our PF-Net, which predicts the missing region of the point cloud from its incomplete known configuration. Fig. 2 shows the complete architecture of our PF-Net. The overall architecture of our PF-Net is composed of three fundamental building blocks, named Multi-Resolution Encoder (MRE), Point Pyramid Decoder (PPD), and Discriminator Network.

3.1. Feature Points Sampling

In a point cloud, we can only take a small number of points to characterize the shape, which are defined as feature points. In other words, feature points map the skeleton of a point cloud [32]. To extract feature points from a point cloud, we use iterative farthest point sampling (IFPS), which is a sampling strategy applied in Pointnet++ [33] to get a set of skeleton points. IFPS can represent the distribution of the entire point sets better compared to random sampling, and it is more efficient than CNNs [33]. In Fig. 3, we visualize the effect of IFPS. Even if we only extract 6.25% of the points, these points can still describe the configuration of the lamp and the main geometry of the lamp has not been destroyed.

3.2. Multi-Resolution Encoder

CMLP We first introduce the feature extractor of our MRE, named as Combined Multi-Layer Perception (CMLP). In most of the previous work, the feature extractor of their encoder is Multi-Layer Perception (MLP). We refer to this as

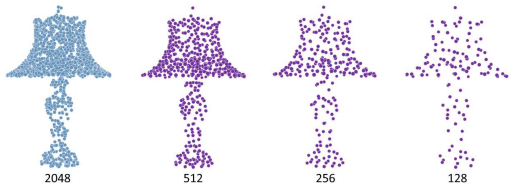


Figure 3. **Results of IFPS in different resolutions.** The blue lamp has 2048 points. Three purple lamps are sampled from the blue lamp with 512, 256, and 128 points, respectively.

PointNet-MLP, shown in Fig. 4(a). This method maps each point into different dimensions and extracts the maximum value from the final K dimensions to form a global latent vector. However, it does not make good use of low-level and mid-level features that contain rich local information. Also, L-GAN [12] and PointNet [10] show that the performance of this feature extractor is strongly affected by the dimension of the Maxpooling layer, i.e., K. In CMLP, we also use MLP to encode each point into multiple dimensions $[64 - 128 - 256 - 512 - 1024]$. Different from MLP, we maxpool the output of the last four layers of MLP to obtain multiple dimensional feature vector f_i, where size f_i := 128, 256, 512, 1024, for $i = 1, ..., 4$. All the f_i are then concatenated, forming the combined latent vector $\mathbf{F}$. Size $\mathbf{F}$:= 1920, and it contains both low-level and high-level feature information.

The input to our Multi-Resolution Encoder is an $N \times 3$ unordered point cloud. We downsample the input point cloud to obtain two more scales in different resolutions (Size: $\frac{N}{k} \times 3$ and $\frac{N}{k^2} \times 3$) by using IFPS. We use this data-dependent way to get the feature points of the input point sets, helping the encoder focus on those more representa-

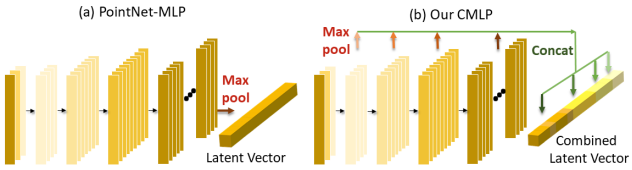


Figure 4. **Comparison of two different feature extractors of 3D point encoders.** PointNet-MLP only uses the final single layer to extract the latent vector. Our CMLP maps the combined latent vector by concatenating multiple dimensional latent vectors derived from the last four layers. It can make better use of the high-level and low-level features.

tive points. Three independent CMLPs will be used to map those three scales into three individual combined latent vector $\mathbf{F}_i$, for $i = 1, 2, 3$. Each vector represents the feature extracted from a certain resolution of the point cloud. All the $\mathbf{F}_i$ are then concatenated, forming a latent feature map $\mathbf{M}$ in the size of 1920×3 ($i.e.$, three vectors each in the size of 1920). We then use MLP [3 - 1] to integrate the latent feature map into a final feature vector $\mathbf{V}$. Size of $\mathbf{V}$=1920.

3.3. Point Pyramid Decoder

The decoder takes final feature vector $\mathbf{V}$ as input and aims to output $M \times 3$ point cloud, which represents the shape of the missing region. The baseline of our PPD is fully-connected decoder [12]. A fully-connected decoder is good at predicting the global geometry of point cloud. Nevertheless, it always causes loss of local geometric information since it only uses the final layer to predict the shape. Previous works combine fully-connected decoder with a folding-based decoder to reinforce the local geometry of the predicted shape. However, as shown in [13], a folding-based decoder is not good at handling genus-wise distortions and retaining the original detailed geometry if the original surface is relatively complicated. To overcome the limitation, we design our PPD as a hierarchical structure based on feature points, which is inspired by FPN [26]. The details of the PPD are shown in Fig. 5. We first obtain three feature layers $\mathbf{FC}_i$ (size $\mathbf{FC}_i$:=1024, 512, 256 neurons, for $i = 1, 2, 3$) by passing $\mathbf{V}$ through fully-connected layers. Each feature layer is responsible for predicting point cloud in different resolutions. The primary center points Y_{primary} will be predicted from the deepest $\mathbf{FC}_3$, having the size of $M_1 \times 3$. Then, $\mathbf{FC}_2$ will be used to predict the relative coordinates of secondary center points $Y_{\text{secondary}}$. Each point in Y_{primary} will serve as center point to generate $\frac{M_2}{M_1}$ points of Y_{primary}. We implement this process using "Expand" and "Add" operations. Thus, size of $Y_{\text{secondary}}$ is $M_2 \times 3$. Detailed points Y_{detail} is the final prediction of PPD. The generation of Y_{detail} is similar to $Y_{\text{secondary}}$ shown in Fig. 5. Size Y_{detail}: $M \times 3$. Meanwhile, Y_{detail} will try to match the feature points sampled from G.T. Thanks to this multi-scale generating architecture, high-level features will affect the expression of low-level features, and low resolution feature points can propagate the local geometry information to the high resolution prediction. Our experiments show that the prediction of PPD has fewer distortions and can retain the detailed geometry of the original missing point cloud.

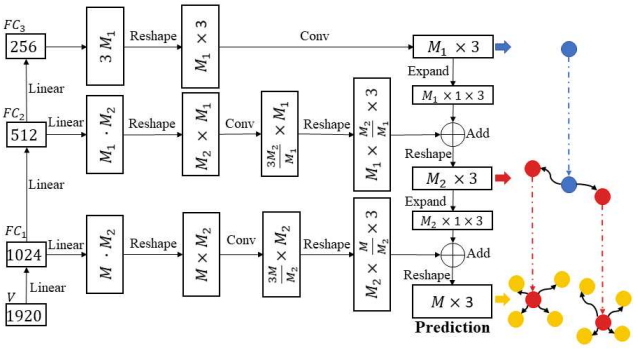


Figure 5. **Details of Point Pyramid Decoder**. PPD gives outputs of three joint layers in ascending scales, which are similar both in semantic and geometric level. Intuitively, the architecture of PPD reflects the concept of "Fractal geometry" in mathematics.

3.4. Loss Function

The loss function of our PF-Net consists of two parts: multi-stage completion loss and adversarial loss. Completion loss measures the difference between the ground truth of missing point cloud Y_{gt} and the predicted point cloud. Adversarial loss tries to make the prediction seem more real by optimizing the MRE and PPD. The size of Y_{gt} is $M \times 3$ which is the same as Y_{detail}.

Multi-stage Completion Loss Fan et al. [34] has proposed two permutation-invariant metrics to compare unordered point cloud which are Chamfer Distance (CD) and Earth Mover's Distance (EMD). In this work, we choose Chamfer Distance as our completion loss since it is differentiable and more efficient to compute compared to EMD.

$$d_{\text{CD}}(S_1, S_2) = \frac{1}{S_1} \sum_{x \in S_1} \min_{y \in S_2} \|x - y\|_2^2 + \frac{1}{S_2} \sum_{y \in S_2} \min_{x \in S_1} \|y - x\|_2^2 \tag{1}$$

CD in (1) measures the average nearest squared distance between predicted point cloud S_1 and the ground truth point cloud S_2. Since the Point Pyramid Decoder will predict three point clouds in different resolution, our multi-stage completion loss consists of three terms in (2), d_{CD_1}, d_{CD_2}, and d_{CD_3}, weighted by hyperparameter α. The first term calculates the squared distance between detailed points Y_{detail} and the ground truth of missing region Y_{gt}. The second and third term calculate the squared distance between primary center points Y_{primary}, secondary center points $Y_{\text{secondary}}$ and the subsampled ground truth Y'_{gt}, Y''_{gt} respectively. The subsampled ground truth Y'_{gt} and Y''_{gt} have the same size as Y_{primary} ($M_1 \times 3$) and $Y_{\text{secondary}}$ ($M_2 \times 3$),

respectively. We obtain $Y'_{\rm gt}$ and $Y''_{\rm gt}$ from $Y_{\rm gt}$ by applying IFPS. They are the feature points of the missing region. The design of the multi-stage completion loss increases the proportion of feature points, leading to better focus on feature points.

$$L_{\rm com} = d_{\rm CD1}(Y_{\rm detail}, Y_{\rm gt}) + \alpha\, d_{\rm CD2}(Y_{\rm primary}, Y'_{\rm gt}) + 2\alpha\, d_{CD3}(Y_{\rm secondary}, Y''_{\rm gt}) \quad (2)$$

Adversarial Loss The adversarial loss in this work is inspired by Generative Adversarial Network (GAN) [35]. We first define $\mathrm{F}() := \mathrm{PPD}(\mathrm{MRE}())$, where MRE is Multi-Resolution Encoder and PPD is Point Pyramid Decoder. $F : \mathcal{X} \rightarrow \mathcal{Y}'$ will map the partial input $\mathcal{X}$ into predicted missing region $\mathcal{Y}'$. Then Discriminator ($\mathrm{D}()$) tries to distinguish the predicted missing region $\mathcal{Y}'$ and the real missing region $\mathcal{Y}$. The discriminator is a classification network with similar structure as CMLP. The specific structure has serial MLP layers $[64-64-128-256]$ and we maxpool the response from the last three layers to obtain feature vector f_i, where size f_i := 64, 128, 256, for i = 1, 2, 3. Concatenate them into a latent vector F. The size of F is 448. F will be passed through fully-connected layers [256,128,16,1] followed by Sigmoid-classifier to obtain predicted value. We now define the adversarial loss:

$$L_{\rm adv} = \sum_{1\le i\le S} \log(D(y_i)) + \sum_{1\le j\le S} \log(1 - D(F(x_i))) \quad (3)$$

where $x_i \in \mathcal{X}, y_i \in \mathcal{Y}$, $i = 1, \ldots, S$. S is the dataset size of $\mathcal{X}, \mathcal{Y}$. Both F and D are optimized jointly by using alternating Adam when training.

Joint Loss The joint loss of our network is defined as:

$$L = \lambda_{\rm com} L_{\rm com} + \lambda_{\rm adv} L_{\rm adv} \quad (4)$$

λ_{com} and λ_{adv} are the weight of completion loss and adversarial loss which satisfy the followed condition: $\lambda_{com} + \lambda_{adv} = 1$.

4. Experiments

4.1. Data Generation and Implementation detail

To train our model, we use 13 categories of different objects in the benchmark dataset Shapenet-Part [36]. The total number of shapes sums to 14473 (11705 for training and 2768 for testing). All input point cloud data is centered at the origin, and their coordinates are normalized to [-1,1]. The ground-truth point cloud data is created by sampling 2048 points uniformly on each shape. The incomplete point cloud data is generated by randomly selecting a viewpoint as a center in multiple viewpoints and removing points within a certain radius from complete data. We control the radius to get a different amount of missing points. When comparing our method against other methods, we set the incomplete point cloud missing 25% of the original data for train and test.

We implement our network on PyTorch. All the three building blocks are trained by using ADAM optimizer alternately with an initial learning rate of 0.0001 and a batch size of 36. We employ batch normalization (BN) and RELU activation units at MRE and Discriminator but only use RELU activation units (except for the last layer) at PPD. In MRE, we set $k = 2$. In PPD, we set $M_1 = 64$, $M_2 = 128$ based on the numbers of points of each shape. And we only change M to control the size of the final prediction.

4.2. Point Cloud Completion Results

In this subsection, we compare our method against several representative baselines that operate directly on a 3D point cloud, including L-GAN [37], PCN [11], 3D point-Capsule Networks [25]. Since all the existing methods are trained in different datasets, we train and test them in the same dataset in order to evaluate them quantitatively. It should be noted that all the methods are trained in the same way that label information will not be provided. To evaluate the methods mentioned above, we use the evaluation metric by [37, 38]. It contains two indexes: Pred $\rightarrow$ GT (prediction to ground truth) error and GT $\rightarrow$ Pred (ground truth to prediction) error. Pred $\rightarrow$ GT error computes the average squared distance from each point in prediction to its closest in ground truth. It measures how difference the prediction is from the ground truth. GT $\rightarrow$ Pred error computes the average square distance from each point in the ground truth to its closest in prediction. which indicates the degree of the ground truth surface being covered by the shape of the prediction. We first compute Pred $\rightarrow$ GT error and GT $\rightarrow$ Pred error on the whole complete point cloud by concatenating the prediction of our network and the input partial point cloud.

Table 1 shows the result. Our method outperforms other methods in all categories on both Pred $\rightarrow$ GT and GT $\rightarrow$ Pred error. Note that the error of the overall complete point cloud comes from two parts: the prediction error of the missing region and the change of the original partial shape. Since our method takes the partial shape as input and only output the missing region, it does not change the original partial shape. To make sure that our evaluation is reasonable, we also compute Pred $\rightarrow$ GT error and GT $\rightarrow$ Pred error on the missing region. Table 2 shows the results. Our Method outperforms existing methods in 12 out of 13 categories on both Pred $\rightarrow$ GT error and GT $\rightarrow$ Pred error. Furthermore, in terms of the mean of all 13 categories, our method has considerable advantages on both metrics. The result in Table 1 and Table 2 indicate that our method can generate more high-precision point cloud with less distortions both in the overall point cloud and the missing region point cloud.

In Fig. 6 we visualize the output point cloud generated by the methods mentioned above. All of them come from

Category	LGAN-AE [12]	PCN [11]	3D-Capsule [25]	PF-Net(vanilla)	PF-Net
Airplane	0.856 / 0.722	0.800 / 0.800	0.826 / 0.881	0.284 / **0.231**	**0.263** /0.238
Bag	3.102 / 2.994	2.954 / 3.063	3.228 / 2.722	0.927 / 0.934	**0.926** / **0.772**
Cap	3.530 / 2.823	3.466 / 2.674	3.439 / 2.844	1.308 / **1.027**	**1.226** / 1.169
Car	2.232 / 1.687	2.324 / 1.738	2.503 / 1.913	0.616 / 0.431	**0.599** / **0.424**
Chair	1.541 / 1.473	1.592 / 1.538	1.678 / 1.563	**0.472** / **0.420**	0.487 / 0.427
Guitar	0.394 / 0.354	0.367 / 0.406	0.298 / 0.461	**0.097** / 0.094	0.108 / **0.091**
Lamp	3.181 / 1.918	2.757 / 2.003	3.271 / 1.912	1.041 / **0.616**	**1.037** / 0.640
Laptop	1.206 / 1.030	1.191 / 1.155	1.276 / 1.254	0.309 / **0.244**	**0.301** / 0.245
Motorbike	1.828 / 1.455	1.699 / 1.459	1.591 / 1.664	0.524 / 0.414	**0.522** / **0.389**
Mug	2.732 / 2.946	2.893 / 2.821	3.086 / 2.961	0.793 / 0.776	**0.745** / **0.739**
Pistol	1.113 / 0.967	0.968 / 0.958	1.089 / 1.086	0.270 / **0.237**	**0.252** / 0.244
Skateboard	0.887 / 1.02	0.816 / 1.206	0.897 / 1.262	0.289 / 0.288	**0.225** / **0.172**
Table	1.694 / 1.601	1.604 / 1.790	1.870 / 1.749	**0.505** / 0.417	0.525 / **0.404**
Mean	1.869 / 1.615	1.802 / 1.662	1.927 / 1.713	0.572 / 0.471	**0.555** / **0.458**

Table 1. **Point cloud completion results of overall point cloud**. The training data consists of 13 categories of different objects [36]. The numbers shown are [Pred → GT error / GT → Pred error], scaled by 1000. We compute the mean values across all categories and show them in the last row of the table. PF-Net (vanilla) is PF-Net without Discriminator.

Category	LGAN-AE [12]	PCN [11]	3D-Capsule [25]	PF-Net(vanilla)	PF-Net
Airplane	3.357 / 1.130	5.060 / 1.243	2.676 / 1.401	1.197 / **1.006**	**1.091** / 1.070
Bag	5.707 / 5.303	**3.251** / 4.314	5.228 / 4.202	3.946 / 4.054	3.929 / **3.768**
Cap	8.968 / 4.608	7.015 / **4.240**	11.04 / 4.739	5.519 / 4.470	**5.290** / 4.800
Car	4.531 / 2.518	2.741 / 2.123	5.944 / 3.508	2.537 / 1.848	**2.489** / **1.839**
Chair	7.359 / 2.339	3.952 / 2.301	3.049 / 2.207	**1.998** / 1.828	2.074 / **1.824**
Guitar	0.838 / 0.536	1.419 / 0.689	0.625 / 0.662	**0.435** / 0.435	0.456 / **0.429**
Lamp	8.464 / 3.627	11.61 / 7.139	9.912 / 5.847	5.252 / **3.059**	**5.122** / 3.460
Laptop	7.649 / 1.413	3.070 / 1.422	2.129 / 1.733	1.291 / 1.013	**1.247** / **0.997**
Motorbike	4.914 / 2.036	4.962 / 1.922	8.617 / 2.708	2.229 / 1.876	**2.206** / **1.775**
Mug	6.139 / 4.735	3.590 / 3.591	5.155 / 5.168	3.228 / 3.332	**3.138** / **3.238**
Pistol	3.944 / 1.424	4.484 / 1.414	5.980 / 1.782	1.267 / **1.012**	**1.122** / 1.055
Skateboard	5.613 / 1.683	3.025 / 1.740	11.49 / 2.044	1.198 / **1.257**	**1.136** / 1.337
Table	2.658 / 2.484	2.503 / 2.452	3.929 / 3.098	**2.184** / **1.928**	2.235 / 1.934
Mean	5.395 / 2.603	4.360 / 2.661	5.829 / 3.008	2.483 / **2.086**	**2.426** / 2.117

Table 2. **Point cloud completion results of the missing point cloud**. The numbers shown are [Pred → GT error / GT → Pred error], scaled by 1000. In this table, We compute those two metrics in the missing region of point cloud.

the test set. Compared to other methods, the prediction of our PF-Net presents strong spatial continuity with a high level of restoration and less genus-wise distortions. [13].

4.3. Quantitative Evaluations of PF-Net

Analysis of Discriminator The function of the Discriminator is to distinguish the predicted shape from the real profile of the missing region and optimize our network to generate a more "realistic" configuration. Table 1 and Table 2 show the result of PF-Net without Discriminator. Compared to PF-Net without Discriminator, PF-Net with Discriminator outperforms in 10 out of 13 categories on the Pred → GT error both in Table 1 and Table 2. Furthermore, PF-Net has a considerable margin in terms of mean on the Pred → GT error both in Table 1 and Table 2. The results indicate that PF-Net can help minimize Pred → GT error. As is mentioned above, Pred → GT error measures how difference the prediction is from the ground truth. Thus, the Discriminator enables PF-Net to generate the point cloud that is more similar to the ground truth.

Analysis of MRE and PPD To demonstrate the effectiveness of CMLP, we train CMLP and other extractors which follow the same linear classification model on ModelNet40 [39] and evaluate their overall classification accuracy on the test set. Results are shown in Table 3. PointNet-MLP has the same parameter as PointNet without Transform-Net [10]. PointNet-CMLP replaces MLP of PointNet-MLP by CMLP. Our-CMLP shows the best performance. The re-

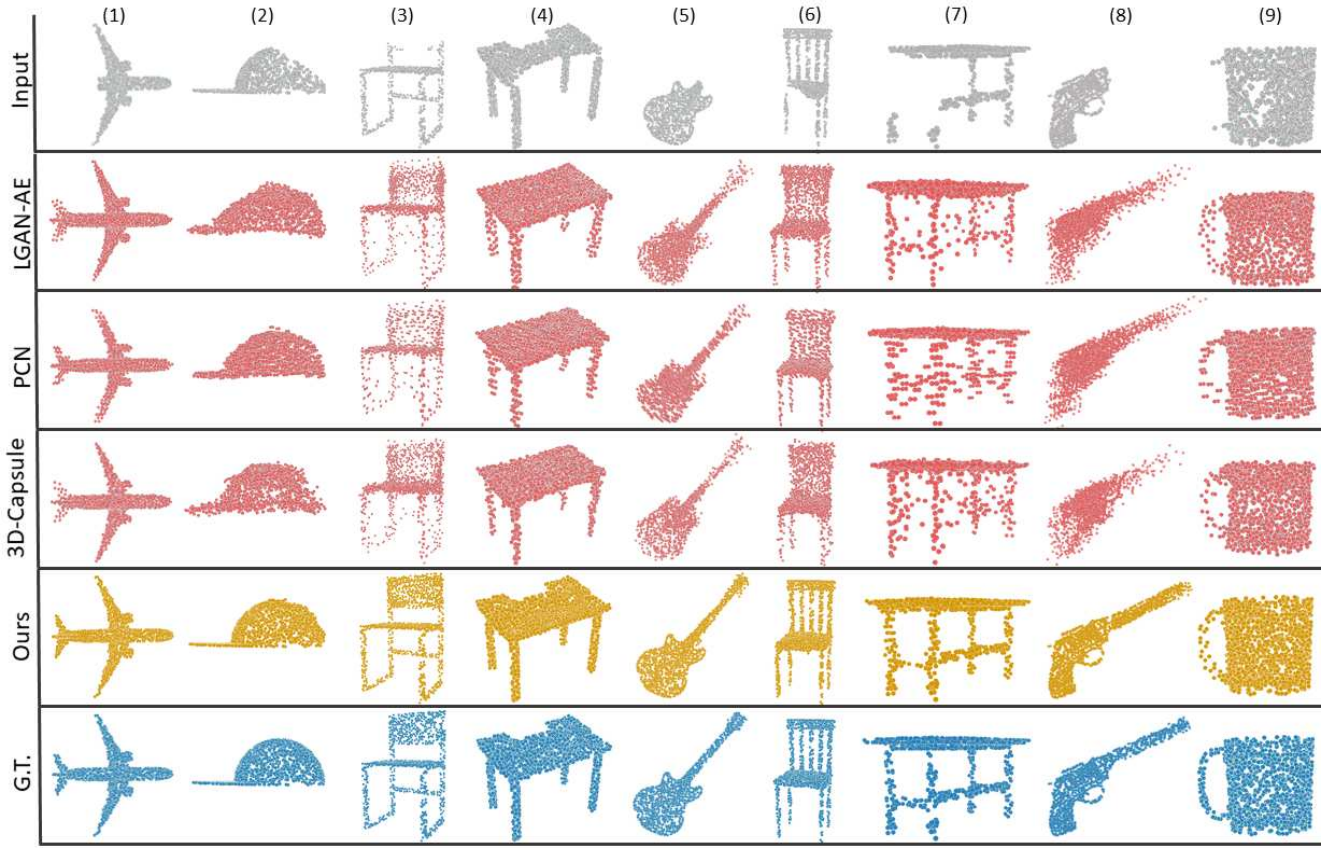


Figure 6. **The comparison of completion results between other methods and our network.** From top to bottom: Input, L-GAN [12], PCN [11], 3D-Capsule [25], our method, and the ground truth. In (3)(7), the outputs of other methods lose the geometric structure of the cross-bar under the chair and the table. In (3)(4)(6), other methods mistakenly fill in the hollow part of the G.T. (i.e., (3)(6), the back of the chair; (4), the space under the tabletop is mistakenly filled). In (8), others fail to generate a reasonable shape. Correspondingly, ours can generate the missing point cloud accurately with less distortions and high-level restoration. More results of ours are shown in Supplement.

sult also indicates that CMLP has a better understanding of semantic information.

	PointNet-MLP	PointNet-CMLP	Our-CMLP
Acc.(%)	87.2	87.9	88.9

Table 3. Classification accuracy results of PointNet-MLP, PointNet-CMLP, and Our-CMLP on ModelNet40.

In order to assess the effectiveness of Multi-Resolution Encoder(MRE) and Point Pyramid Decoder(PPD), we further compare our PF-Net(vanilla) with three different baselines: single-resolution MLP, single-resolution CMLP, and multi-resolution CMLP. All the above methods take the incomplete point cloud as input and output the missing point cloud. We conduct this experiment on the models in "Chair" and "Table" class, which are two largest categories in the dataset we generated to discuss. "Chair" has 2658 shapes for training and 704 shapes for testing. "Table" has 3835 shapes for training and 848 shapes for testing. Single-resolution MLP consists of 5 layers MLP [64,128,256,512,1024] and 2 linear layers [1024,1536], each layer followed by BN and ReLU activation (i.e we find that BN is beneficial in this method). Single-resolution CMLP has the same structure as single-resolution MLP but MLP is changed to CMLP. Multi-resolution CMLP use multi-resolution encoder followed by 2 linear layers [1024, 1536]. We compute their Pred$\rightarrow$ GT error and GT $\rightarrow$ Pred error of the missing point cloud. Results are shown in Table 4. Note that, in both two categories, CMLP and MR-CMLP are superior to MLP in Pred $\rightarrow$ GT error and GT $\rightarrow$ Pred error. Their performance boost is between 3% to 10% compared to MLP. However, after using PF-Net, the Pred $\rightarrow$ GT error and GT $\rightarrow$ Pred error are reduced significantly. This is what we expected. Since CMLP and MR-CMLP strengthen the ability of encoder to extract geometrical and semantic information, but their decoders only focus on generating the overall shape of the missing point cloud.

However, the generation process of PPD is smoother. It can focus on both the overall shape and the detailed feature points. Fig. 7 depicts one example of the "chair" class.

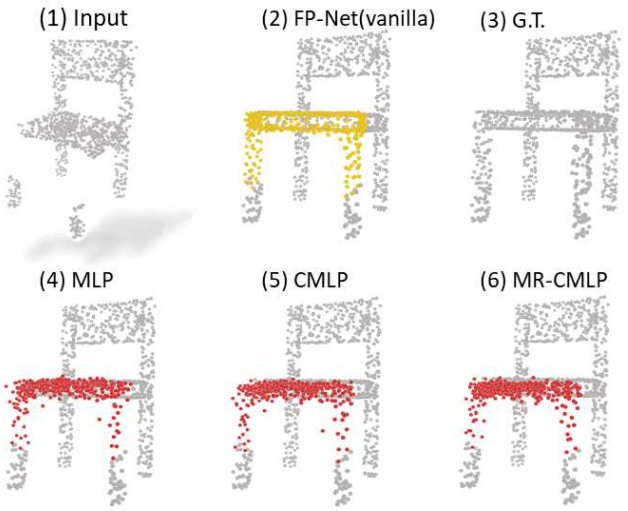


Figure 7. **A chair predicted by MLP, CMLP , MR-CMLP, and PF-Net(vanilla)**. PF-Net(vanilla) is our PF-Net without Discriminator. Yellow and red points represent the prediction of PF-Net(vanilla) and others. It can be found that the middle of the chair in the G.T. is hollow. The prediction of MLP, CMLP, and MR-CMLP all lose this detailed structure. Our PF-Net(vanilla) retains this structure, and the prediction has less noise and distortions.

Category	MLP	CMLP	MR-CMLP	PF-Net(vanilla)
Chair	2.692 / 2.384	2.587 / 2.306	2.521 / 2.275	**1.772** / **1.432**
Table	2.429 / 2.734	2.250 / 2.693	2.299 / 2.696	**1.891** / **1.664**

Table 4. [Pred $\rightarrow$ GT error / GT $\rightarrow$ Pred error] given by MLP, CMLP, MR-CMLP and our PF-Net(vanilla) of the missing point cloud.

Robustness Test We conduct all the robustness test experiments on the "Airplane" class. In the first robustness test, we change M in the PPD to control the number of the output points of our network and train it to repairing incomplete shape with different extent of incompletion, respectively. The experiment results are shown in Table 5: 25%, 50% and 75% mean that three incomplete inputs lose 25%, 50% and 75% of the original points respectively compared to the ground truth. Note that the Pred $\rightarrow$ GT error and GT $\rightarrow$ Pred error of three partial inputs are basically the same, which implies that our network has strong robustness when dealing with incomplete inputs with different extent of missing. Fig. 8 shows the performance of our network in the test set. Our network accurately "identified" different types of airplane and retain the geometric details of the original point cloud even in the case of large-scale incompletion. In the second robustness test, we train our network to complete the partial shape that lose more than one pieces of points in different positions. One of the example in test set is shown in Fig. 9. It can be found that PF-Net still predict the correct missing point cloud in the right places.

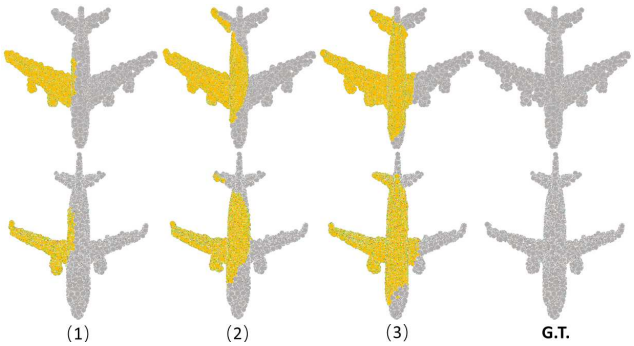


Figure 8. **Examples of repairing results when the input has different extent of incompletion**. (1), (2), (3) lose 25%, 50%, 75% points of the original point cloud respectively. Yellow represents the prediction. Grey denotes the undamaged point cloud.

Missing Ratio	25%	50%	75%
PF-Net	0.727 / 0.542	0.727 / 0.520	0.748 / 0.623

Table 5. [Pred $\rightarrow$ GT error / GT $\rightarrow$ Pred error] given by our PF-Net of the missing point cloud. The incomplete point cloud lose 25%, 50%, 75% respectively compared to the original point cloud.

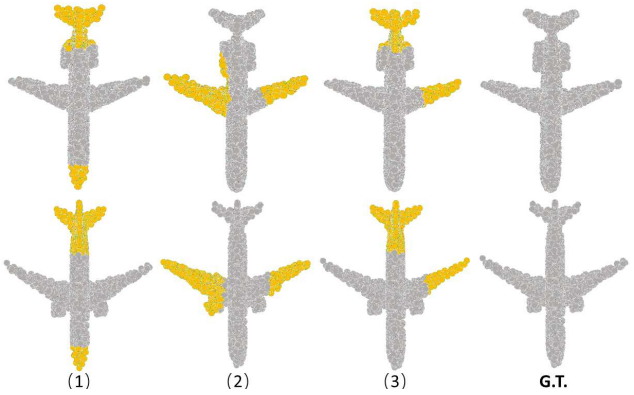


Figure 9. Repairing results when the original point cloud loses different numbers of points in two random positions. Yellow represents the prediction. Grey denotes the undamaged point cloud.

5. Conclusion

We have presented a new approach to shape completion using a partial point cloud only to generate the missing point cloud. Our architecture enables the network to generate the target point cloud with both rich semantic profile and detailed character while retaining the existing contour. Our method outperforms the existing methods focusing on shape completion of a point cloud. Noticing that if the training dataset is as large enough, our method has the chance to repair any complex random point cloud delicately. Hence, we can foresee that if this method is applied deeply, the accuracy of 3D recognition can be dramatically improved, bringing new possibilities to the research of autonomous vehicles and 3D reconstruction.

References

[1] X. Zhang, X. Le, A. Panotopoulou, E. Whiting, and C. C. Wang, "Perceptual models of preference in 3D printing direction," *ACM Trans. Graphics*, vol. 34, no. 6, pp. 1–12, 2015.

[2] X. Zhang, X. Le, Z. Wu, E. Whiting, and C. C. Wang, "Data-driven bending elasticity design by shell thickness," in *Computer Graphics Forum*, vol. 35, no. 5, 2016, pp. 157–166.

[3] B. Zhao, X. Le, and J. Xi, "A novel sdass descriptor for fully encoding the information of a 3D local surface," *Information Sciences*, vol. 483, pp. 363–382, 2019.

[4] Y. Liu, B. Fan, S. Xiang, and C. Pan, "Relation-shape convolutional neural network for point cloud analysis," *CVPR*, 2019.

[5] Y. Feng, H. You, Z. Zhang, R. Ji, and Y. Gao, "Hypergraph neural networks," *AAAI*, 2019.

[6] T. Yu, J. Meng, and J. Yuan, "Multi-view harmonized bilinear network for 3D object recognition," *CVPR*, 2018.

[7] X. He, Y. Zhou, Z. Zhou, S. Bai, and X. Bai, "Triplet-center loss for multi-view 3D object retrieval," *CVPR*, 2018.

[8] A. Kanezaki, Y. Matsushita, and Y. Nishida, "RotationNet: Joint object categorization and pose estimation using multiviews from unsupervised viewpoints," *CVPR*, 2018.

[9] Y. Zhou and O. Tuzel, "VoxelNet: End-to-end learning for point cloud based 3D object detection," *CVPR*, 2018.

[10] C. R. Qi, H. Su, K. Mo, and L. J. Guibas, "Pointnet: Deep learning on point sets for 3D classification and segmentation," *CVPR*, 2017.

[11] W. Yuan, T. Khot, D. Held, C. Mertz, and M. Hebert, "PCN: Point completion network," *3DV*, 2018.

[12] P. Achlioptas, O. Diamanti, I. Mitliagkas, and L. J. Guibas, "Learning representations and generative models for 3D point clouds," *ICML*, 2018.

[13] Y. Yang, C. Feng, Y. Shen, and D. Tian, "Foldingnet: Point cloud auto-encoder via deep grid deformation," *CVPR*, 2018.

[14] M. Sarmad, H. Lee, and Y. M. Kim, "RL-GAN-Net: A reinforcement learning agent controlled GAN network for real-time point cloud shape completion," *CVPR*, 2019.

[15] D. Pathak, P. Krahenbuhl, J. Donahue, T. Darrell, and A. A. Efros, "Context encoders: Feature learning by inpainting," *CVPR*, 2016.

[16] B. Yang, H. Wen, S. Wang, R. Clark, A. Markham, and N. Trigoni, "3D object reconstruction from a single depth view with adversarial learning," *ICCV Workshops*, 2017.

[17] A. Dai, C. R. Qi, and M. Niebner, "Shape completion using 3D-Encoder-Predictor CNNs and shape synthesis," *CVPR*, 2017.

[18] W. Wang, Q. Huang, S. You, C. Yang, and U. Neumann, "Shape inpainting using 3D generative adversarial network and recurrent convolutional networks," *ICCV*, 2017.

[19] C. Li, M. Zaheer, Y. Zhang, B. Poczos, and R. Salakhutdinov, "Point cloud GAN," *ICLR workshops*, 2018.

[20] X. Han, . Z. Li, and . H. Huang, "High-resolution shape completion using deep neural networks for global structure and local geometry inference," in *ICCV*, 2017.

[21] J. Wu, C. Zhang, T. Xue, W. T. Freeman, and J. B. Tenenbaum, "Learning a probabilistic latent space of object shapes via 3D generative-adversarial modeling," *NeurIPS*, 2016.

[22] Y. Yu, Z. Huang, F. Li, H. Zhang, and X. Le, "Point encoder gan: A deep learning model for 3d point cloud inpainting," *Neurocomputing*, 2019.

[23] L. Yu, X. Li, C. Fu, D. Cohenor, and P. Heng, "PU-Net: Point cloud upsampling network," *CVPR*, 2018.

[24] S. Sabour, N. Frosst, and G. E. Hinton, "Dynamic routing between capsules," *NeurIPS*, 2017.

[25] Y. Zhao, T. Birdal, H. Deng, and F. Tombari, "3D point capsule networks," *CVPR*, 2018.

[26] T. Y. Lin, P. Dollar, R. Girshick, K. He, B. Hariharan, and S. Belongie, "Feature pyramid networks for object detection," *CVPR*, 2017.

[27] K. Chen, J. Pang, J. Wang, Y. Xiong, X. Li, S. Sun, W. Feng, Z. Liu, J. Shi, W. Ouyang *et al.*, "Hybrid task cascade for instance segmentation," *CVPR*, 2019.

[28] Z. Tian, C. Shen, H. Chen, and T. He, "Fcos: Fully convolutional one-stage object detection," *CVPR*, 2019.

[29] J. Pang, K. Chen, J. Shi, H. Feng, W. Ouyang, and D. Lin, "Libra R-CNN: Towards balanced learning for object detection," *CVPR*, 2019.

[30] Z. Cai and N. Vasconcelos, "Cascade R-CNN: Delving into high quality object detection," *CVPR*, 2018.

[31] A. Kirillov, R. Girshick, K. He, and P. Dollr, "Panoptic feature pyramid networks," *CVPR*, 2019.

[32] T. Birdal and S. Ilic, "A point sampling algorithm for 3D matching of irregular geometries," *IROS*, 2017.

[33] C. R. Qi, L. Yi, H. Su, and L. J. Guibas, "Pointnet++: Deep hierarchical feature learning on point sets in a metric space," *NeurIPS*, 2017.

[34] H. Fan, S. Hao, and L. Guibas, "A point set generation network for 3D object reconstruction from a single image," *CVPR*, 2017.

[35] I. Goodfellow, J. Pougetabadie, M. Mirza, B. Xu, D. Wardefarley, S. Ozair, A. Courville, and Y. Bengio, "Generative adversarial nets," *NeurIPS*, 2014.

[36] L. Yi, V. G. Kim, D. Ceylan, I. Shen, M. Yan, H. Su, C. Lu, Q. Huang, A. Sheffer, and L. J. Guibas, "A scalable active framework for region annotation in 3D shape collections," *ACM Trans. on Graphics*, vol. 35, no. 6, pp. 210:1–210:12, 2016.

[37] C. Lin, C. Kong, and S. Lucey, "Learning efficient point cloud generation for dense 3D object reconstruction," *AAAI*, 2018.

[38] M. Gadelha, R. Wang, and S. Maji, "Multiresolution tree networks for 3D point cloud processing," *ECCV*, 2018.

[39] Z. Wu, S. Song, A. Khosla, F. Yu, L. Zhang, X. Tang, and J. Xiao, "3D shapenets: A deep representation for volumetric shapes," *CVPR*, 2015.